

SWEN90004 Assignment 2 Proposal: Schelling's Segregation Model

Group Members:

- Jinzheng Xiang — 1698063
- Qi He — 1314559
- Tong Zhao — 1463847

1. Model Overview

This model simulates the behavior of two types of agents in a neighborhood, blue and orange which get along with each other but like to live with similar agents, moving in the neighborhood due to this. The result of this movement causes segregation between the two types of agents. This model is useful for social scientists to study housing patterns in cities and other social systems.

2. Original Model Design

The original model has agents which are either blue or orange on a 2d grid, each occupying exactly one square, not sharing with any other agent, with the two parameters of "DENSITY" which controls the total number of agents and "%-SIMILAR-WANTED" which controls the minimum amount of same coloured agents which the agent wants to be neighbours with in order to be "HAPPY".

The initial positions of the agents are randomly generated at setup. Each Agent can be happy or unhappy, each agent also tracks the number of similar and different agents nearby, and the total number of agents nearby.

The model outputs "%SIMILAR" which is the average percentage of similar nearby agents across all agents, "NUM-UNHAPPY" which is the total number of unhappy agents, and "%UNHAPPY", the percentage of agents that are unhappy. These are the global attributes.

UPDATE RULES(PER TICK):

1. Check if all agents are happy, if so, then stop updating, if not, then continue.
2. Move unhappy agents, agents moving sequentially in a random order. For each agent:
 - a. the agent turns a random amount and moves a random amount;
 - b. if that square is unoccupied, then the agent stops there, if not, loop
3. Update each agent's attributes. For each agent:
 - a. count amount of similar agents nearby;
 - b. count amount of different agents nearby;
 - c. count total amount of agents nearby;
 - d. check if should be happy based on %-SIMILAR-WANTED;
 - e. visualise;
4. Update global attributes.
 - a. update %SIMILAR
 - b. update NUM-UNHAPPY and %UNHAPPY

3. Our Java Implementation Design

We will implement the model in **Java SE 26** using only the standard library, following an object-oriented design.

Core classes:

Class	Attributes	Methods	Responsibility
Agent	current_position, colour happy	isHappy(List<Agent> neighbours, double percentSimilarWanted), updatePosition(Position newPos)	Holds colour, current grid position, and computes is_happy() given the local neighbourhood.

Grid	cells, width, height	getNeighbours(x, y), findEmptyPatches(), moveAgent(Agent agent, Position pos), getAgentAt(x, y)	2D array of agent references (or None for empty patches). Provides get_neighbours(x, y), and find_empty_patches().
Simulation	grid, density, percentSimilarWanted, currentTick, isCoverged	setup(), step(), calculateGlobalMetrics(), writeMetricsToCSV()	Owns the Grid, manages parameters, runs setup() and step(), and writes per-tick metrics to a CSV file.
Experiment Runner	densityRange, similarityRange, numReplications	runPhase1Replication(), runPhase2Extension(), aggregateStatistics(), reportSummaryCSV()	Sweeps parameter ranges, performs repeated runs for each configuration, and aggregates statistics.

Key methods:

- Simulation.setup(): populates the grid by placing an agent on each patch with probability density, assigning colour uniformly at random.
- Simulation.step(): identifies unhappy agents, relocates each to a random empty patch (preserving NetLogo's asynchronous semantics), then recomputes happiness and global metrics.
- Grid.get_neighbours(x, y): returns up to 8 occupied neighbours.

Output: each run produces a CSV file with columns tick, %similar, %unhappy, num_unhappy. Aggregate statistics across runs are written to a separate summary CSV.

4. Planned Experiments

Phase 1 — Replication: We will compare our implementation against the NetLogo model across the following scenarios:

1. **Baseline reproduction:** density = 95%, %-similar-wanted = 30%. Verify that at equilibrium the %similar converges to approximately 75% like the netlogo model.
2. **Sweeping** %-similar-wanted: range 0–80% in 10% increments, fixed density = 95%. Measure final % similar and convergence time.
3. **Sweeping** density: range 50–95% in 5% increments, fixed %-similar-wanted = 30%. Identify the regime where the model fails to converge.

For each parameter setting, we will run 3 times and compare the distribution of outcomes between the two implementations.

Phase 2 — Extension: We propose to extend the model by introducing a moving cost mechanism. In the original model, unhappy agents always relocate at no cost, which assumes residential mobility is frictionless. In reality, moving incurs costs that may deter relocation for dissatisfied residents. We can model this by assigning each agent a personal dissatisfaction tolerance and only allowing relocation when the agent's same-colour neighbour deficit exceeds a configurable cost threshold.

This allows us to address the question of "How does the introduction of a moving cost affect the equilibrium level of segregation, and is there a cost threshold above which segregation is substantially suppressed compared to the cost-free baseline?"

Experiments will sweep the moving-cost parameter across a range of values and compare the resulting segregation level (% similar), the proportion of agents that remain unhappy at equilibrium, and the total number of relocations against the cost-free baseline.

We will also examine whether moving cost interacts with %-similar-wanted — for instance, whether a small cost is sufficient to prevent segregation under low intolerance but ineffective under high intolerance.

5. Task Breakdown and Timeline

Week	Date Range	Member 1	Member 2	Member 3
1	8 May – 12 May	NetLogo exploration & baseline runs	Implement Agent, Grid classes	Implement Simulation.setup() and step()
2	13 May – 18 May	Verify implementation against NetLogo (Phase 1)	Build ExperimentRunner and CSV output	Run replication experiments and aggregate results
3	19 May – 23 May	Implement moving-cost extension	Design and run extension experiments	Statistical analysis and figure generation
4	24 May – 28 May	Draft Background & Replication sections	Draft Extension & Discussion sections	Code documentation, formatting check, references
Final	29 May	Group review, proof-reading, submission		

6. AI Acknowledgement

In preparing this proposal, some members of our group used GenAI to assist with understanding the assignment requirements, exploring possible extension directions for the Schelling model. All design decisions, model choices, experimental plans, and task allocations were made and verified by group members. Code implementation has not yet begun at this stage; AI use during the implementation and report-writing phases will be documented in the final report.